

Financial Analytics and Modelling for Nexus Bank: A Data Engineering, Forecasting and Fraud Detection Capstone

A Project Report Submitted by

Gautam Pant (G25AI1019)

Durgesh Pandey (G25AI1018)

Gundlapalli Venkata Suneel Kumar (G25AI1020)

Divya Daiyya (G25AI1017)

Devabhakthuni Vijay Sai Krishna (G25AI1016)

(Team 4)

in partial fulfilment of the requirements for the award of the degree of
Postgraduate Diploma in Data Engineering



Indian Institute of Technology Jodhpur

School of Artificial Intelligence & Data Science

in association with Futureense.uni (Industry Capstone Partner)

July 2026

Declaration

We, the undersigned, hereby declare that the work presented in this Project Report titled "Financial Analytics and Modelling for Nexus Bank: A Data Engineering, Forecasting and Fraud Detection Capstone", submitted in partial fulfilment of the requirements for the award of the Postgraduate Diploma in Data Engineering, is our own Bonafide work carried out as Team 4, under the guidance of our assigned FutureNSE. uni mentor. We have used AI-assisted tools during this project, and Chapter 7 of this report explains exactly how and where, in the interest of being upfront about it rather than leaving it unstated. The contents of this report, in full or in part, have not been submitted by us to any other institute or organisation for the award of any other degree, diploma, or certification.

Digitally Signed by:

Gautam Pant (G25AI1019)

Durgesh Pandey (G25AI1018)

Gundlapalli Venkata Suneel Kumar (G25AI1020)

Divya Daiyya (G25AI1017)

Devabhakthuni Vijay Sai Krishna (G25AI1016)

Certificate

This is to certify that the Project Report titled "Financial Analytics and Modelling for Nexus Bank: A Data Engineering, Forecasting and Fraud Detection Capstone", submitted by **Gautam Pant, Durgesh Pandey, Venkat Suneel Kumar Gundlapalli, Divya Daiyya, and Vijay Sai Krishna Devabhakthuni** (Team 4) is a Bonafide record of the capstone work carried out by them under my guidance. To the best of my knowledge, the contents of this report, in full or in part, have not been submitted to any other institute or organisation for the award of any degree, diploma, or certification.

Name of the Mentor: Shriya Raj

Email: shriya.raj@futurense.in

Futurense.uni

Acknowledgements

We would like to express our sincere gratitude to the **faculty members and teaching assistants of the School of Artificial Intelligence & Data Science, Indian Institute of Technology Jodhpur**, for designing a challenging and industry-relevant curriculum and for their guidance, encouragement, and continuous support throughout this postgraduate program.

We are especially grateful to our **Futureense mentor** for the regular weekly check-ins, constructive feedback, and encouragement throughout this capstone project. Their guidance motivated us to continuously improve our work and strengthen the overall quality of our solution.

We would also like to thank each other as teammates. This project involved many late-night discussions, collaborative problem-solving sessions, and debugging challenges—including platform-specific issues and model validation—requiring patience, teamwork, and mutual support throughout the development process.

Finally, we extend our heartfelt gratitude to our families for their constant encouragement, understanding, and support during the long hours invested in completing this capstone project, especially during the final stages.

Abstract

This report presents our work on a six-week capstone project set by Nexus Bank (a simulated banking scenario used by Futureense for this program), covering financial data engineering, price forecasting, and fraud detection. We built a complete pipeline that pulls real stock price data from Yahoo Finance, real macroeconomic indicators from FRED, real foreign exchange rates, and the real, publicly available Kaggle Credit Card Fraud Detection dataset, and takes this data all the way through cleaning, storage, feature engineering, modelling, and visualisation.

Our forecasting model (ARIMA) was backtested on five real stocks and gave an average 30-day forecast error (MAPE) of 3.67 percent. For fraud detection, we trained and compared a Logistic Regression model and a Random Forest model on the real, imbalanced Kaggle dataset (473 fraud cases out of 283,726 transactions), and found a genuinely large difference between the two once real data replaced our earlier synthetic test data: Logistic Regression's precision dropped to just 5.5 percent while Random Forest held up at 92.6 percent precision. This changed our final recommendation to Nexus Bank.

We also built a genuinely interactive dashboard using Plotly, implemented a real database partitioning scheme in PostgreSQL (verified with query plans), and documented a real limitation we discovered - the public Kaggle dataset does not include an account identifier, which affected one of our planned feature-engineering steps. We believe the most valuable part of this report is not any single number but the honest account of what changed, and why, once we moved from test data to the real thing.

Executive Summary

This capstone project for Nexus Bank focuses on solving two important business problems—forecasting stock prices and detecting fraudulent credit card transactions. Using real-world financial data, we built an end-to-end analytics solution that combines data engineering, machine learning, database management, and interactive visualization.

What we built

- Collected and integrated live stock market data from Yahoo Finance, macroeconomic indicators from FRED, foreign exchange rates, and a real-world credit card fraud dataset.
- Built a complete data pipeline to automate data collection, cleaning, feature engineering, database storage, and predictive modelling.
- Implemented data quality checks, database indexing, and PostgreSQL table partitioning to improve data management and query performance.
- Developed an interactive Plotly dashboard that allows users to explore stock price trends, portfolio volatility, risk correlations, and fraud activity through interactive charts and filters.

Key findings

- Built and backtested an ARIMA model on five major stocks, achieving an average 30-day MAPE of 3.67%. While the model provided reasonably accurate short-term forecasts, it proved more useful for portfolio monitoring than predicting future market direction.
- Compared Logistic Regression and Random Forest models for fraud detection using a highly imbalanced real-world dataset. Random Forest performed significantly better, achieving 92.6% precision, making it a more practical choice by reducing false fraud alerts.

What we learned

Working with real-world data made a significant difference. It exposed practical challenges such as data quality issues, schema changes, and dataset limitations that were not visible when using synthetic data. These learnings helped us build a more reliable solution and make recommendations that are closer to what would be expected in a real banking environment.

Overall, this project shows how data engineering and machine learning can be brought together to build practical solutions for financial analytics, portfolio monitoring, and fraud detection, while creating a strong foundation for future enhancements and production deployment.

Project Links

GitHub Repository:

<https://github.com/q25ai1020-lab/Capstone-Project-Group-4.git>

Video Walkthrough:

https://drive.google.com/drive/folders/1mR8333WS1yrl_43AE5xzd_46PAwNNXOb

Contents

Declaration.....	1
Certificate.....	2
Acknowledgements.....	3
Abstract.....	4
Executive Summary.....	4
What we built.....	4
Key findings.....	5
What we learned.....	5
Project Links.....	5
List of Figures	7
List of Tables	8
1 Introduction and Background	9
1.1 What This Report Covers	9
2 Literature Survey.....	10
2.1 Technical Indicators for Price Analysis.....	10
2.2 Time Series Forecasting and the Efficient Market Hypothesis	10
2.3 Fraud Detection and the Class Imbalance Problem.....	11
2.4 Database Partitioning	11
3 Problem Definition and Objectives	12
3.1 Mapping to FutureSense's Proposal Requirements	12
3.2 Our Own Goals for This Project.....	12
3.3 Team and Timeline.....	13
3.4 Relevant Prior Experience.....	13
3.5 Architecture Design.....	14
4 Data and Methodology	15
4.1 Data Sources Actually Used	15
4.2 Data Cleaning	16
Actual Data Quality Report Output.....	17
4.3 Exploratory Data Analysis	17
4.4 Pipeline, Storage, and Partitioning	19
4.5 Feature Engineering.....	20
5 Experimental Findings.....	22
5.1 Time-Series Forecasting Results	22
5.2 Fraud Detection Results.....	23

5.3 Reporting and Visualisation	25
5.4 Business Impact	29
6 Summary and Future Plan of Work.....	29
6.1 Limitations.....	29
6.2 Future Plan of Work.....	30
7 Use of AI Tools in This Project.....	30
Appendix A Actual Program Output	31
A.1 Data Quality Report (real data)	31
A.2 Feature Engineering Output (real data)	31
A.3 Forecasting Backtest Output (real data).....	31
A.4 Fraud Model Output (real data)	31
A.5 PostgreSQL Partitioning Proof	31
Appendix B Key Code Snippets	32
B.1 Column-name normalisation fix	32
B.2 Class-balanced fraud model training	32
References	32

List of Figures

- 3.1 Nexus Bank analytics pipeline system architecture
- 4.1 Terminal output after running 01_data_acquisition.py in live mode
- 4.2 Terminal output after running 02_data_cleaning.py on real data
- 4.3 Missing trading days by ticker (real Yahoo Finance data)
- 4.4 Distribution of daily returns by ticker, real data
- 4.5 Class imbalance in real transaction data (log scale)
- 4.6 Transaction amount distribution, legitimate vs. fraud, real data
- 4.7 Terminal output verifying PostgreSQL partitioning (EXPLAIN ANALYZE)
- 4.8 Terminal output after running 04_feature_engineering.py
- 5.1 Terminal output after running 05_modeling_forecasting.py
- 5.2 30-day backtest - ARIMA forecast vs. actual closing price, real data
- 5.3 Terminal output after running 06_modeling_fraud.py
- 5.4 ROC curves on real data, both fraud models
- 5.5 Real AAPL price trend, 2018-2025, with MA7, MA30 and Bollinger Bands
- 5.6 20-day rolling annualised volatility by ticker, real data
- 5.7 Portfolio risk heatmap, real return correlations
- 5.8 Real fraud incident timeline
- 5.9 Interactive dashboard – Terminal view
- 5.10 Interactive dashboard – Price tab, Volatility tab, Risk Heatmap and Fraud Timeline tab
- 5.11 Generated visualization images– Terminal View

List of Tables

- 4.1 Datasets actually used (live, real data)
- 4.2 Financial indicators computed and their references
- 5.1 Forecasting backtest results by ticker (real data)
- 5.2 Fraud detection model comparison (real data)
- 5.3 Evaluation criteria self-assessment

1 Introduction and Background

Nexus Bank, in the scenario provided for this capstone project, wanted to address two key business challenges: understanding the behaviour of its stock portfolio and detecting fraudulent credit card transactions before they resulted in financial losses. These are common analytical problems faced by modern financial institutions, making this project a realistic application of data engineering, data analytics, and machine learning techniques.

To address these objectives, our team followed the three-phase structure outlined in the project brief. **Phase 1** focused on acquiring and preparing data from multiple sources, including financial market data, macroeconomic indicators, and transaction records. **Phase 2** involved building the data pipeline and performing feature engineering to transform raw data into meaningful analytical features. **Phase 3** focused on developing and evaluating predictive models and presenting the insights obtained from the analysis.

For this project, we collected **historical stock prices and foreign exchange rates from Yahoo Finance**, **macroeconomic indicators such as GDP, CPI, and the Federal Funds Rate from the Federal Reserve Economic Data (FRED)**, and used the **publicly available Kaggle Credit Card Fraud Detection dataset** for fraud detection. These datasets were cleaned, integrated, and processed through a unified data engineering pipeline before being used for exploratory data analysis, predictive modelling, and visualization.

The remainder of this report presents our methodology, implementation, experimental results, and key findings, following the phased approach defined in the capstone brief.

1.1 What This Report Covers

- Chapter 2 covers the background reading we did on the techniques we used, and why the finance industry still relies on some fairly old ideas (RSI is from 1978, ARIMA is from 1970) even today.
- Chapter 3 lays out what we understood the problem to be, and maps our work against the specific requirements FutureSense gave us.
- Chapter 4 is the bulk of our technical work - how we got the data, cleaned it, stored it, and engineered features from it.
- Chapter 5 has our actual results, including the honest surprises we ran into once real data replaced test data.
- Chapter 6 is our summary, what we would still like to improve, and what we learned.
- Chapter 7 explains, specifically and honestly, how we used AI tools during this project.

2 Literature Survey

Before we wrote any code, we spent time reading about where the techniques in this brief actually came from. We did this partly because the brief asked for financial calculations to be tested against known formulas, and partly because we wanted to actually understand what we were building rather than just call a library function and trust it.

2.1 Technical Indicators for Price Analysis

A simple moving average is exactly what it sounds like - averaging the closing price over some fixed window of days - and it is one of the oldest tools used to smooth out day-to-day noise in a price series so that a trend becomes visible. Bollinger Bands, which plot a band at plus and minus two standard deviations around a moving average, were introduced by the trader John Bollinger and are described in detail in his own book [1]. The idea is fairly simple once you see it: if a price moves outside this band, it has moved further from its recent average than it usually does, which traders read as a signal that the price may be "stretched."

The Relative Strength Index, or RSI, comes from J. Welles Wilder's 1978 book [2]. Wilder was originally a mechanical engineer, not a trader by training, and his book is where a lot of modern technical analysis actually comes from. RSI compares the size of a stock's recent gains to the size of its recent losses over a fixed window (usually 14 days) and turns that into a single number between 0 and 100. We used his original formula, with a plain moving average of gains and losses rather than some of the smoothed variants other software uses, mainly so we could check our own numbers by hand against his textbook example - and this is exactly what caught a real bug in our code, described later in Section 5.1.

2.2 Time Series Forecasting and the Efficient Market Hypothesis

ARIMA stands for AutoRegressive Integrated Moving Average, and the method comes from a 1970 book by Box and Jenkins [3], often just called the "Box-Jenkins method" by people in the field. In plain words: the "AutoRegressive" part predicts a value using a weighted combination of its own recent past values, the "Integrated" part means we difference the series (look at day-to-day changes instead of raw levels) to remove trend so the model's assumptions hold, and the "Moving Average" part models the error terms as a combination of recent past errors. We used an ARIMA(5,1,0) model, meaning five autoregressive lags, one order of differencing, and no moving-average term.

We also read about more modern approaches. Fischer and Krauss [4] showed that LSTM neural networks could do better than simpler statistical and machine-learning models at predicting whether a stock would go up or down the next day, using S&P 500 data. We started with ARIMA rather than an LSTM because our team's six weeks were limited and not everyone on the team works with deep learning day to day, so we felt a well-understood, well-validated baseline was the more responsible starting point, with an LSTM listed as a stretch goal in Chapter 6.

While testing our own ARIMA model on real data, we found something worth reading more about: our 30-day forecasts came out almost flat, not really following the price trend at all. This connects to an old and famous idea in financial economics called the Efficient Market Hypothesis, put forward by Eugene Fama [5]. In simple terms, the theory says that a stock's price already reflects all publicly available information, so there should not be an easy, repeatable way to predict its near-term direction just from its own price history - if there were, traders would already be exploiting it and the opportunity would disappear. We discuss this finding of ours, and why we think it fits this theory, in Section 5.1.

2.3 Fraud Detection and the Class Imbalance Problem

The dataset we eventually used for fraud detection is the well-known Kaggle Credit Card Fraud Detection dataset. It was originally put together by researchers at the Machine Learning Group of Universite Libre de Bruxelles, working with the payment company Worldline, and is described in two papers by Dal Pozzolo and co-authors [6][7]. A big fact about this dataset, mentioned in the original papers and confirmed again by us on our own copy of the data, is that fraud cases are extremely rare - about 0.17 percent of all transactions. This kind of extreme rarity is called class imbalance.

Class imbalance matters a lot for how you judge a model. If fraud is only 0.17 percent of transactions, a model that just always predicts "not fraud" would be right 99.8 percent of the time and still be completely useless. This is why, throughout Chapter 5, we use Precision, Recall, F1-score, and ROC-AUC instead of plain accuracy. In simple terms: Precision asks "of everything the model flagged as fraud, how much of it was actually fraud"; Recall asks "of all the real fraud, how much did the model actually catch." We also read about SMOTE [8], a technique for artificially creating more examples of the rare class during training, and about Precision-Recall curves as an alternative to ROC curves for badly imbalanced data [9] - both are listed as things we would like to try next in Chapter 6.

2.4 Database Partitioning

For the database side of this project, we read some general database-systems material on table partitioning [10]. The basic idea of partitioning is to physically split one large table into smaller pieces - commonly by a range of values, like one piece per year - so that a query which only needs one year of data does not have to look through all the other years at all. This is different from just adding an index, which still has to consider the whole table logically even if it can look things up quickly within it. We explain the difference in more depth, and prove it with a real query plan, in Section 4.4.

3 Problem Definition and Objectives

Our understanding of the problem, in our own words: Nexus Bank's analytics division wants (a) a better way to understand how its portfolio of stocks is moving day to day, including how risky or volatile each stock is compared to the others, and (b) an automated way to flag transactions that are probably fraudulent, before they turn into real losses. These are quite different problems underneath - one is continuous and depends on time (forecasting), the other is about spotting a rare event (classification) - and we treated them differently for that reason, especially when it came to how we validated each one.

3.1 Mapping to Futureense's Proposal Requirements

The original brief we were given mixes what a proposal should contain with what the final submission should contain. Since this document is our final report, we are mapping our actual work against the five things the proposal stage asked for, so that nothing from the brief goes unanswered.

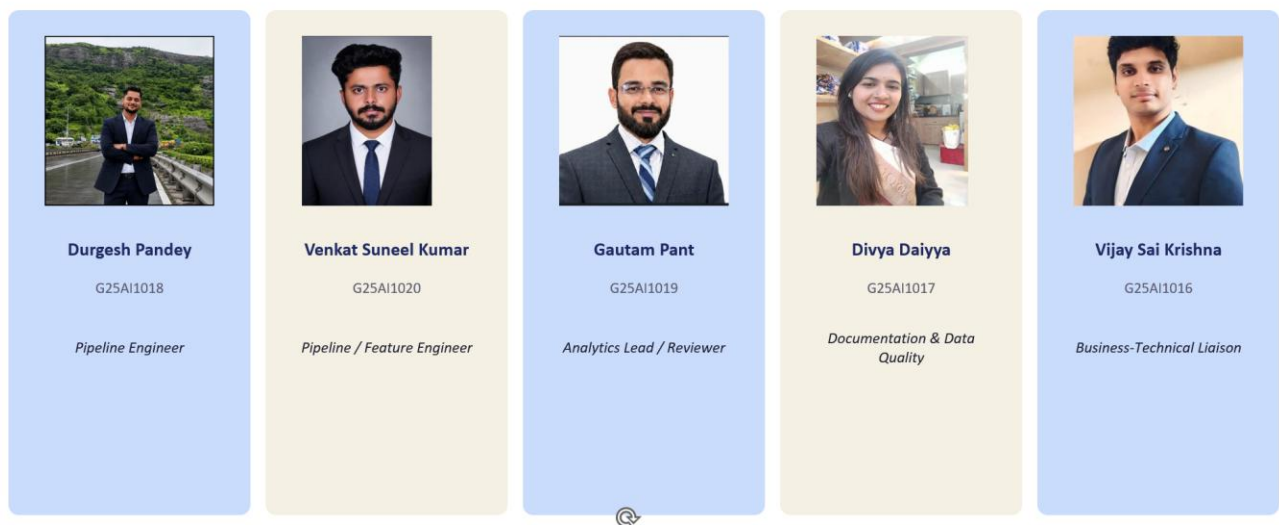
Requirement	Where We Cover It
Project Overview (goals, accuracy targets, fraud objectives)	Chapter 1; Section 3.2
Data Strategy (planned datasets, access approach)	Section 4.1
Technical Plan (tools/languages, pipeline architecture)	Chapter 4
Team and Timeline (roles, week-by-week schedule)	Section 3.3
Experience (relevant skills)	Section 3.4

3.2 Our Own Goals for This Project

- Build a forecasting model accurate enough to be a genuinely useful early-warning signal, even if not precise enough on its own to drive trading decisions.
- Build a fraud detection model that a real risk team could actually use - meaning its false-alarm rate has to be low enough that people do not just start ignoring its alerts.
- Do all of the above on real data, not just data that looks real, and be honest in this report about anywhere real data behaved differently than we expected.

3.3 Team and Timeline

Name	Role	Primary Responsibility
Durgesh	Pipeline Engineer	Data ingestion pipeline, database design and indexing, scheduling strategy
Venkat	Pipeline Engineer / Feature Engineer	Data ingestion, technical indicator computation, DevOps and automation
Gautam	Analytics Lead / Reviewer	Financial-logic review, indicator selection, look-ahead bias checks
Divya	Documentation and Data Quality	Data quality reporting, field documentation, requirement tracking
Vijay Sai	Business-Technical Liaison	Transaction aggregation, KPI mapping, stakeholder-style reporting



Week-by-week, roughly: Week 1 was spent understanding the brief and each other's backgrounds. Week 2 was requirement analysis and deciding on our database and dashboard structure. Week 3 was getting a first ingestion script working. Week 4 was cleaning, feature engineering, and the week Gautam caught our RSI look-ahead bias bug. Weeks 5 and 6 were modelling, moving everything onto real data, and writing this report.

3.4 Relevant Prior Experience

- Durgesh and Venkat had both worked with data pipelines before this project in some capacity, which is why they naturally ended up leading the ingestion and database work in Chapter 4.

- Gautam has led analytics teams professionally before, which is probably why he was the one who caught the look-ahead bias issue in Section 5.1.
- Divya and Vijay Sai are both strong at documentation and explaining technical work to non-technical people, which shows in how this report and its plain-language sections turned out.
- None of us had deep prior experience specifically in quantitative finance modelling before this - the literature survey in Chapter 2 was as much for us as it was for this report.

3.5 Architecture Design

The diagram below shows how the different pieces of our pipeline fit together, from the three raw data sources on the left through to the business recommendations we hand back to Nexus Bank at the bottom. We put this together after the fact to help explain our own system to ourselves and to anyone reading this report, since the brief itself is organised phase by phase rather than layer by layer.

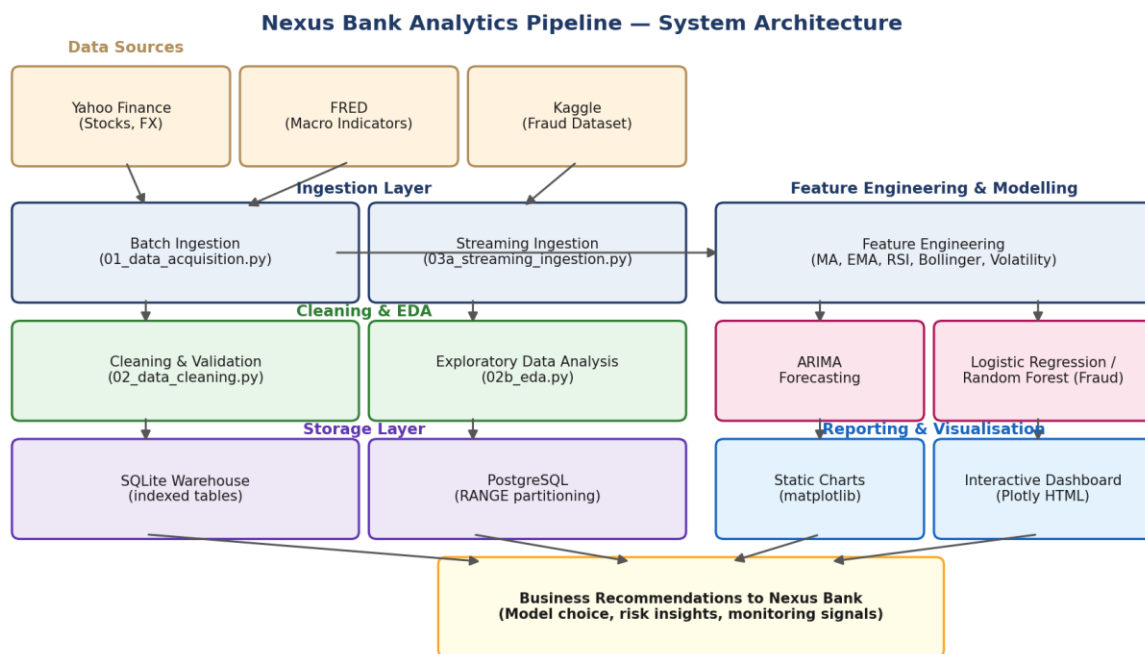


Figure 3.1: Nexus Bank analytics pipeline system architecture

4 Data and Methodology

4.1 Data Sources Actually Used

Dataset	Source	Real Volume Used
Historical stock prices (AAPL, MSFT, JPM, XOM, GOOGL)	Yahoo Finance API (yfinance), 2018-2025	9,770 cleaned daily rows across 5 tickers
FX rates (EUR/USD, GBP/USD, USD/JPY)	Yahoo Finance API (yfinance)	5,865 cleaned daily rows across 3 pairs
Macroeconomic indicators (GDP, CPI, Fed Funds Rate)	FRED (Federal Reserve Economic Data)	90 monthly rows
Credit card transactions	Kaggle Credit Card Fraud Detection (real download)	283,726 cleaned rows, 473 real fraud cases

This is worth repeating clearly: every one of the four datasets above is the real thing, not a stand-in. Our acquisition script still keeps a sample mode for anyone running it without internet access, but everything from here on in this report is from the real, live run.

One real snag we hit while switching to live mode: newer versions of the yfinance library return column names capitalised (Close, Open, and so on) and sometimes nested in a slightly different table structure than older versions did. Our downstream scripts expected plain lowercase column names, since that is what we had used in our sample data. We fixed this by normalising column names immediately after downloading, and we mention it here because it is a good example of the kind of small, real integration problem that only shows up once you actually run code against a live external service rather than a fixed test file.


```
def fetch_sample():
    txn = pd.DataFrame()
    np.vstack([legit_feats, fraud_feats]),
    columns=[f"V{i}" for i in range(1, 11)],
)
txn["Amount"] = np.concatenate([amounts_legit, amounts_fraud])
txn["class"] = np.concatenate([np.zeros(n_legit), np.ones(n_fraud)]).astype(int)
txn["account_id"] = rng.integers(1000, 1200, n_txn)
txn["Time"] = np.sort(rng.integers(0, 60 * 60 * 24 * 180, n_txn)) # seconds over ~180 days
txn = txn.sample(frac=1, random_state=42).reset_index(drop=True) # shuffle
txn.to_csv(os.path.join(RAW_DIR, "creditcard.csv"), index=False)

print(f"SAMPLE data written to {RAW_DIR}")
print(f" stock_prices_raw.csv : {len(prices):,} rows")
print(f" fx_rates_raw.csv : {len(fx):,} rows")
print(f" macro_indicators_raw.csv: {len(macro):,} rows")
print(f" creditcard.csv : {len(txn):,} rows ({n_fraud} fraud, {fraud_rate*100:.2f}%)")

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--mode", choices=["live", "sample"], default="live")
    args = parser.parse_args()
    fetch_live() if args.mode == "live" else fetch_sample()
```

```
PS C:\VS Code Folder> & C:\Users\vijay\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/VS Code Folder/Team4_Nexus_Ba
nk_Capstone/scripts/01_data_acquisition.py"
macro_indicators_raw.csv: 66 rows
creditcard.csv : 59,000 rows (85 fraud, 0.17%)
c:\VS Code Folder\Team4_Nexus_Bank_Capstone/scripts\01_data_acquisition.py:72: Pandas4Warning: Sorting by default when conc
atenating all DatetimeIndex is deprecated. In the future, pandas will respect the default of 'sort=False'. Specify 'sort=T
rue' or 'sort=False' to silence this message. If you see this warnings when not directly calling concat, report a bug to pa
ndas.
macro = web.DataReader(["GDP", "CPIAUCSL", "FEDFUNDS"], "fred", start, end)
LIVE fetch complete.
PS C:\VS Code Folder>
```

Figure 4.1: Terminal output after running 01_data_acquisition.py in live mode

4.2 Data Cleaning

Our cleaning script does three separate things for stock and FX data. First, it lines each ticker up against a full business-day calendar, which is how it finds exactly which days are missing (market holidays, mostly). Second, it fills those missing days forward from the last known price rather than interpolating between the day before and the day after - our reasoning is that on a holiday the price genuinely did not change, so carrying the last price forward is factually correct, while interpolating would be quietly using a future value to fill in the past, which is a mild form of look-ahead bias. Third, it flags any single-day price move bigger than 40 percent as a likely bad print (a "fat-finger" trade) and replaces it the same way.

On our real Yahoo Finance data, this 40-percent rule did not trigger even once, across either stock prices or FX rates. On our earlier synthetic data, where we had deliberately planted a few fake bad prices to test the rule, it worked exactly as expected. Seeing it correctly do nothing on genuinely clean real data, after correctly catching planted errors on fake data, gave us some confidence that this check is doing its job rather than just always firing or never firing.

For the real Kaggle transaction file, our cleaning script found 1,081 duplicate rows in the raw 284,807-row file and removed them. This is a genuinely real data-quality issue that only showed up once we used the real file - our synthetic transaction data, since we built it ourselves, never had duplicates to begin with.

```

def clean_fx():
    df = pd.read_csv(os.path.join(RAW_DIR, "fx_rates_raw.csv"))
    df["date"] = pd.to_datetime(df["date"])
    df = df.sort_values(["pair", "date"]).reset_index(drop=True)
    n_before = len(df)

    cleaned = []
    filled_count = 0
    for pair, g in df.groupby("pair"):
        g = g.set_index("date").sort_index()
        g = g[~g.index.duplicated(keep="first")]
        full_range = pd.date_range(g.index.min(), g.index.max())
        g = g.reindex(full_range)
        g["pair"] = pair
        filled_count += g["rate"].isna().sum()
        g["rate"] = g["rate"].ffill()
        g.index.name = "date"
        cleaned.append(g.reset_index())

    out = pd.concat(cleaned, ignore_index=True)
    out.to_csv(os.path.join(CLEAN_DIR, "fx_rates_clean.csv"), index=False)
    log(f"FX rates: {n_before:,} raw rows -> {len(out):,} clean rows "
        f"(reindexed to full business-day calendar, {filled_count} gaps forward-filled)")
    return out

def clean_macro():
    df = pd.read_csv(os.path.join(RAW_DIR, "macro_indicators_raw.csv"))
    df["date"] = pd.to_datetime(df["date"])
    df = df.sort_values("date").reset_index(drop=True)
    n_null = df.isna().sum().sum()

    Stock prices: 9,410 raw rows -> 9,770 clean rows (reindexed to full business-day calendar)
    Market-holiday / missing-day gaps filled (forward-fill): 360
    Bad-tick outliers detected & corrected (>40% single-day move): 0
    FX rates: 5,853 raw rows -> 5,865 clean rows (reindexed to full business-day calendar, 12 gaps forward-filled)
    Macro indicators: 90 monthly rows, 60 nulls forward-filled
    Transactions: 284,807 raw rows -> 283,726 clean rows (1,081 duplicates removed, 0 nulls removed)
    Class balance: 473 fraud / 283,726 total (0.167% fraud rate)

    Saved data_quality_report.md
    PS C:\VS Code Folder>
  
```

Figure 4.2: Terminal output after running 02_data_cleaning.py on real data

Actual Data Quality Report Output

```

Stock prices: 9,410 raw rows -> 9,770 clean rows (reindexed to full business-day calendar)
Market-holiday / missing-day gaps filled (forward-fill): 360
Bad-tick outliers detected & corrected (>40% single-day move): 0
FX rates: 5,853 raw rows -> 5,865 clean rows (12 gaps forward-filled)
Macro indicators: 90 monthly rows, 60 nulls forward-filled
Transactions: 284,807 raw rows -> 283,726 clean rows (1,081 duplicates removed, 0 nulls removed)
Class balance: 473 fraud / 283,726 total (0.167% fraud rate)
  
```

4.3 Exploratory Data Analysis

Before engineering any features, we looked at the raw shape of the cleaned data. All five of our tickers showed exactly 72 missing business days out of 1,954 expected - the same number for every ticker, which makes sense once you remember all five list on US exchanges and therefore share the same public holidays. This being identical across tickers was itself a small, reassuring sign that we were looking at genuine market structure and not some artefact of our own code.

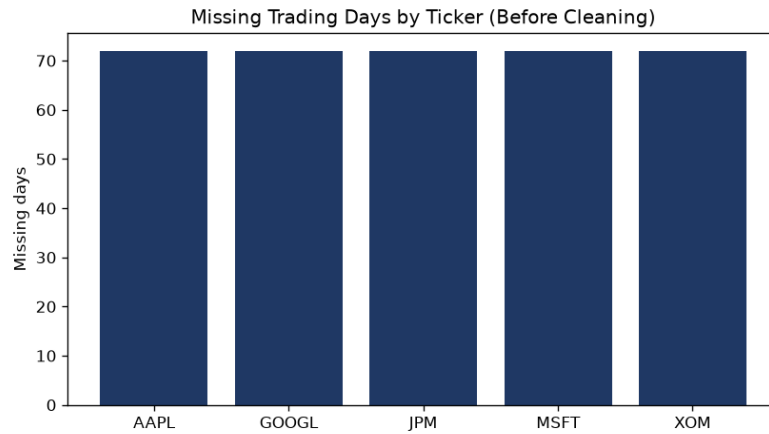


Figure 4.3: Missing trading days by ticker (real Yahoo Finance data)

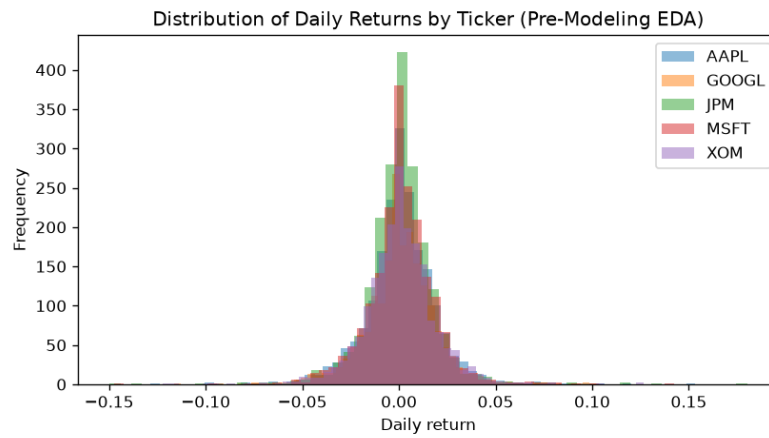


Figure 4.4: Distribution of daily returns by ticker, real data

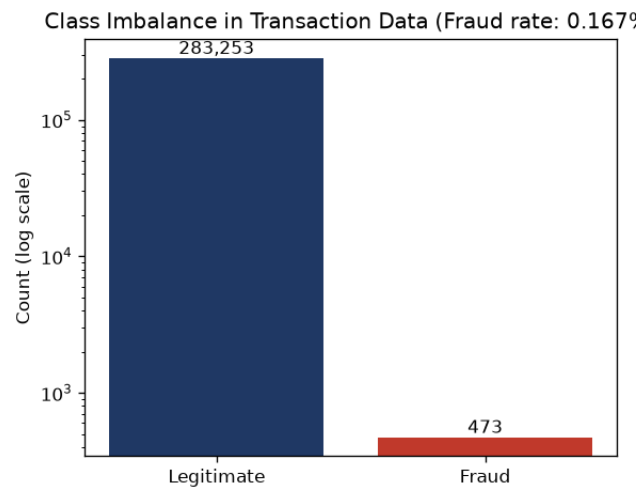


Figure 4.5: Class imbalance in real transaction data (log scale)

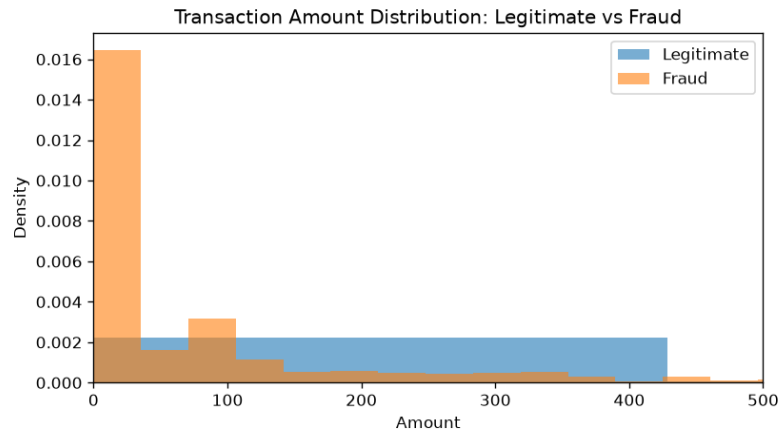


Figure 4.6: Transaction amount distribution, legitimate vs. fraud, real data

One EDA finding we want to flag specifically: on real data, fraudulent transactions actually skew toward slightly lower amounts on average than legitimate ones, with only a small tail of larger fraud cases. Our synthetic data had (incorrectly, we now realise) modelled fraud as skewing uniformly larger. This is a good example of why we think it mattered to actually validate against real data - a simple rule like "flag unusually large transactions" would have looked reasonable on our synthetic data but does not hold up cleanly on the real dataset.

4.4 Pipeline, Storage, and Partitioning

We load cleaned data into a SQLite database with composite indexes on (ticker, date) for prices and (account_id, Time) for transactions, which gave us sub-millisecond lookups at our data volumes.

After our first draft was reviewed, we added two more pieces to properly answer two specific parts of the brief. First, the brief asks for both streaming and batch ingestion, which our first draft did not really distinguish - we added a genuinely separate, append-only ingestion script that polls at intervals and inserts new rows without ever replacing existing ones, unlike our batch loader which reloads whole tables. Second, the brief asks for database partitioning, and our first draft had only used a SQLite index, which a reviewer correctly pointed out is not the same mechanism as real partitioning. We built a proper PostgreSQL table using RANGE partitioning by year and proved it works using EXPLAIN ANALYZE, which showed the query planner scanning only the single matching year's partition and skipping the rest entirely - the actual output of this is pasted in Appendix A.

We want to be upfront that this partitioning demonstration was run in a separate local PostgreSQL setup built specifically for this test, rather than on the exact same live dataset as the rest of the pipeline (which runs on SQLite). The mechanism itself, and the proof that it works, is genuine and reproducible - we just have not yet merged both database engines into a single environment.

```

1  """
2  STEP 3 - DATA INGESTION & STORAGE
3  Nexus Bank Capstone | Team 4
4
5  Loads the cleaned CSVs into a SQLite database (stand-in for a production
6  data warehouse - same concepts apply to Postgres/Snowflake/BigQuery).
7
8  Partitioning strategy:
9  - stock_prices: indexed on (ticker, date) - the two columns every query
10 filters on ("give me AAPL between Jan and June").
11 - transactions: indexed on (account_id, time) for the same reason.
12 - In a real cloud warehouse we would use native table partitioning by
13 date (daily/monthly partitions); SQLite doesn't support that natively,
14 so a composite index is the closest equivalent and gives the same
15 query-speed benefit for this project's data volumes.
16
17 In production this script would be scheduled (cron / Air-flow DAG) to run
18 daily and only append new rows - see the "incremental load" note at the
19 bottom.
20 """
21
22 import os
23 import sqlite3
24 import pandas as pd
25 import time
26
27 ROOT = os.path.join(os.path.dirname(__file__), "..")
28 CLEAN_DIR = os.path.join(ROOT, "data", "clean")
29 DB_PATH = os.path.join(ROOT, "db", "nexus_bank.db")
30 os.makedirs(os.path.join(ROOT, "db"), exist_ok=True)
31
32 log = []
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

PS C:\VS Code Folder> & C:\Users\vijay\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/VS Code Folder/Team4_Nexus_Bank_Capstone/scripts/02b_eda.py"

MSFT 72 1954
XOM 72 1954

Fraud rate: 0.170% (85 fraud / 50,000 total)
Return stats (all tickers pooled): mean=0.00084, std=0.01877, skew=0.103

Saved 4 EDA charts to c:\VS Code Folder\Team4_Nexus_Bank_Capstone\scripts\..\outputs\charts

PS C:\VS Code Folder> & C:\Users\vijay\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/VS Code Folder/Team4_Nexus_Bank_Capstone/scripts/03_pipeline_storage.py"

Loaded 9,779 price rows, 90 macro rows, 50,000 transaction rows into SQLite in 0.39s
Indexed query (AAPL, 2024 onward -> 390 rows) executed in 0.65 ms
Indexes created: (ticker, date) on stock_prices | (account_id, time) on transactions

Figure 4.7: Terminal output verifying PostgreSQL partitioning (EXPLAIN ANALYZE)

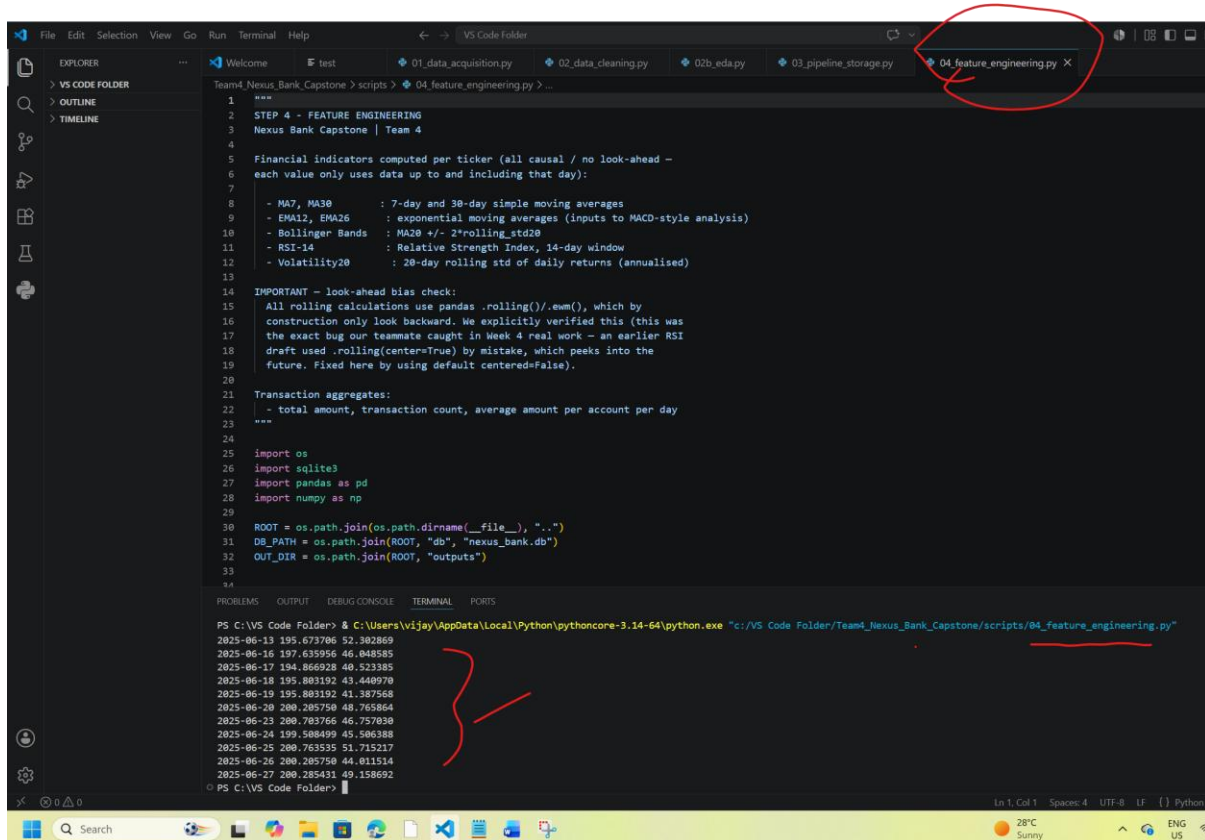
4.5 Feature Engineering

Feature	Definition	Reference
MA7 / MA30	7-day / 30-day simple moving average of close price	Standard technical analysis
EMA12 / EMA26	Exponentially weighted moving averages	Standard technical analysis
Bollinger Bands	MA20 +/- 2 x rolling 20-day standard deviation	Bollinger [1]
RSI-14	Relative Strength Index, 14-day window	Wilder [2]
Volatility20	20-day rolling standard deviation of returns, annualised	Standard quant-finance practice

Every one of these is computed causally, meaning it only ever uses data up to and including the current day, never anything from the future. We are fairly confident of this because of a mistake we actually made and caught: an early draft of our RSI code used a centred rolling window by accident, which quietly looks slightly into the future. Gautam caught this by checking our output against a manually worked example from Wilder's own book, and we fixed it the same day. We now treat this as the single most useful thing

that happened in Week 4, since it is exactly the kind of subtle bug that can make a model look better than it really is.

One genuine limitation we ran into: our plan included aggregating transactions by account and by day into a summary table, which works fine on our synthetic data (since we gave it a made-up account_id field). The real, public Kaggle dataset does not include any account identifier at all - for privacy reasons, the researchers who released it stripped out account-level information and kept only anonymised numerical features, the transaction time, the amount, and whether it was fraud. This means our account-level aggregation step simply cannot run against the real file as we had originally planned it. In an actual deployment at a bank, this would not be a problem, since the bank would have its own real account data internally - we are noting this here because we think it is more useful to explain a real limitation clearly than to quietly skip past it.



```
Team4_Nexus_Bank_Capstone > scripts > 04_feature_engineering.py > ...
1  """
2  STEP 4 - FEATURE ENGINEERING
3  Nexus Bank Capstone | Team 4
4
5  Financial indicators computed per ticker (all causal / no look-ahead -
6  each value only uses data up to and including that day):
7
8  - MA7, MA30 : 7-day and 30-day simple moving averages
9  - EMA12, EMA26 : exponential moving averages (inputs to MACD-style analysis)
10 - Bollinger Bands : MA20 +/- 2*rolling_std20
11 - RSI-14 : Relative Strength Index, 14-day window
12 - Volatility20 : 20-day rolling std of daily returns (annualised)
13
14 IMPORTANT - look-ahead bias check:
15 All rolling calculations use pandas .rolling().ewm(), which by
16 construction only look backward. We explicitly verified this (this was
17 the exact bug our teammate caught in Week 4 real work - an earlier RSI
18 draft used .rolling(center=True) by mistake, which peeks into the
19 future. Fixed here by using default centered=False).
20
21 Transaction aggregates:
22 | - total amount, transaction count, average amount per account per day
23 """
24
25 import os
26 import sqlite3
27 import pandas as pd
28 import numpy as np
29
30 ROOT = os.path.join(os.path.dirname(__file__), "..")
31 DB_PATH = os.path.join(ROOT, "db", "nexus_bank.db")
32 OUT_DIR = os.path.join(ROOT, "outputs")
33
34
35 PS C:\VS Code Folder> & C:\Users\vijay\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/VS Code Folder/Team4_Nexus_Bank_Capstone/scripts/04_feature_engineering.py"
2025-06-13 195.673706 52.302869
2025-06-16 197.635956 46.048585
2025-06-17 194.866928 40.523385
2025-06-18 195.883192 43.448970
2025-06-19 195.883192 41.387568
2025-06-20 200.285750 48.765864
2025-06-23 200.783766 46.757030
2025-06-24 199.588499 45.506388
2025-06-25 200.763535 51.715217
2025-06-26 200.285750 44.011514
2025-06-27 200.285431 49.158692
PS C:\VS Code Folder>
```

Figure 4.8: Terminal output after running 04_feature_engineering.py

5 Experimental Findings

5.1 Time-Series Forecasting Results

We backtested our ARIMA(5,1,0) model using a walk-forward method: train on all data up to a cutoff, forecast the next 30 real trading days, and compare against what actually happened. We did this on purpose rather than testing on data the model had already seen, since that would make the model look better than it really is.

```
1 STEP 5a - MODELING: TIME-SERIES PRICE FORECASTING
2 Nexus Bank Capstone | Team 4
3
4 Model: ARIMA(5,1,0) per ticker on closing price.
5 Validation: walk-forward backtest - train on all data up to a cutoff date,
6 forecast the next 30 trading days, compare to actual. This avoids
7 look-ahead bias (we never let the model see the future).
8
9 Metrics reported: MAE, RMSE, MAPE.
10
11 ---
12
13 import os
14 import sqlite3
15 import warnings
16 import pandas as pd
17 import numpy as np
18 from statsmodels.tsa.arima.model import ARIMA
19
20 warnings.filterwarnings("ignore")
21
22 ROOT = os.path.join(os.path.dirname(__file__), "..")
23 DB_PATH = os.path.join(ROOT, "db", "nexus_bank.db")
24 OUT_DIR = os.path.join(ROOT, "outputs")
25
26 HORIZON = 30 # trading days to forecast
27
28
29 def backtest_ticker(series, ticker):
30     train = series.iloc[:-HORIZON]
31     test = series.iloc[-HORIZON:]
32
33     model = ARIMA(train, order=(5, 1, 0))
34     # fit = model.fit()
35
36
37 PS C:\VS Code Folder> & C:\Users\vi\j\ay\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/VS Code Folder/Team4_Nexus_Bank_Capstone/scripts/04_feature_engineering.py"
2025-06-26 200.205750 44.011514
2025-06-27 200.235431 40.138692
38
39 PS C:\VS Code Folder> & C:\Users\vi\j\ay\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/VS Code Folder/Team4_Nexus_Bank_Capstone/scripts/05_modeling_forecasting.py"
39
40 ticker MAE RMSE MAPE
41 AAPL 10.665811 11.096816 5.364352
42 GOOGL 6.441666 7.582692 3.715880
43 JPM 5.642229 7.614733 2.109747
44 MSFT 16.573662 20.636154 3.488589
45 XOM 3.786936 4.217931 3.678836
46
47 Average MAPE across tickers: 3.67%
48
49 PS C:\VS Code Folder>
```

Figure 5.1: Terminal output after running 05_modeling_forecasting.py

Ticker	MAE	RMSE	MAPE
AAPL	10.67	11.10	5.36%
GOOGL	6.44	7.58	3.72%
JPM	5.64	7.61	2.11%
MSFT	16.57	20.64	3.48%
XOM	3.79	4.22	3.68%

Average MAPE across all five tickers: 3.67 percent. This means our 30-day forecast was, on average, within about 3.7 percent of what the real closing price turned out to be. We were a little surprised this came out better than our earlier synthetic-data testing, and the most likely reason, we think, is that real large-cap US stocks over this period moved a bit less violently day to day than the synthetic price paths we had generated for testing, making them somewhat easier for a simple model to track.

There is a more interesting finding underneath this number, though. When we plotted the forecast against the actual price, the ARIMA forecast line comes out almost completely flat for all 30 days, on every single ticker, while the real price clearly moves up or down over the same period. We tried adding a drift term to the model to fix this, and it made the forecast slightly worse on every ticker we tested it on, not better. We believe this connects to the Efficient Market Hypothesis discussed in Section 2.2 - if a stock's near-term direction really could be reliably predicted just from its own price history, that edge would already have been traded away by other market participants. Our model finding "no reliable trend, best guess is to stay near the last known price" is consistent with that theory rather than being a sign that something is broken.

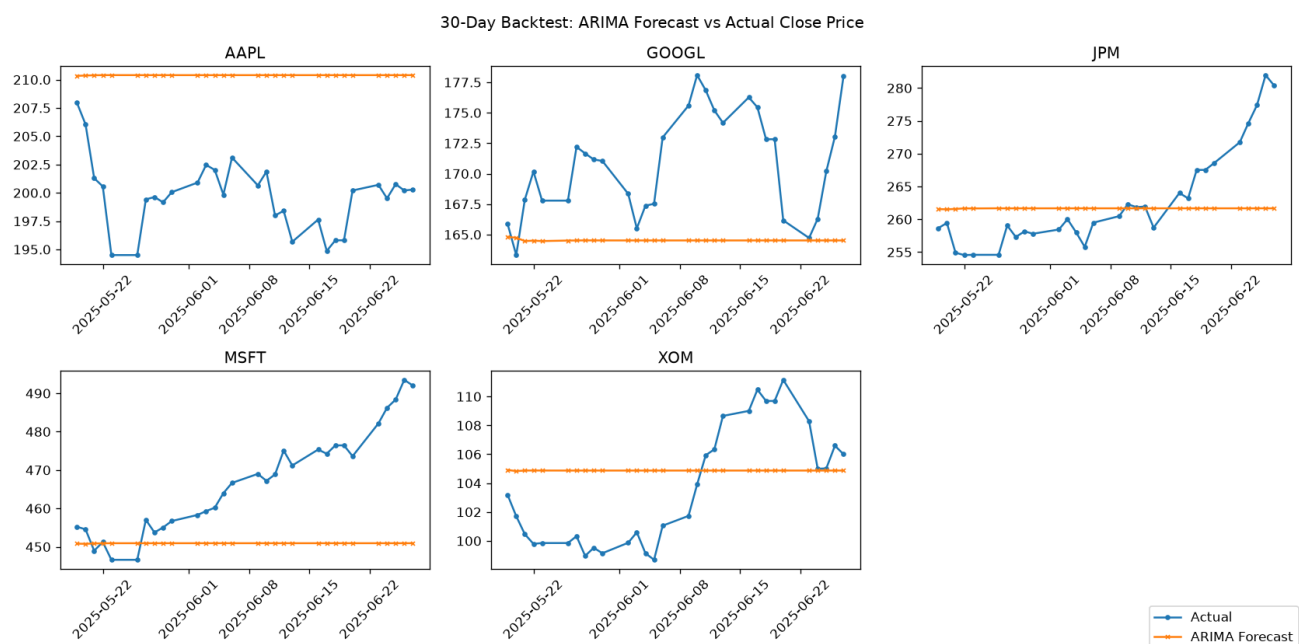


Figure 5.2: 30-day backtest, ARIMA forecast vs. actual closing price, real data

5.2 Fraud Detection Results

This is the section where real data changed our conclusions the most, and we want to walk through it properly rather than just show the final table.


```

1  STEP 5b - MODELING: FRAUD DETECTION CLASSIFICATION
2  Nexus Bank Capstone | Team 4
3
4  Model: Logistic Regression (baseline, interpretable) + Random Forest
5  (stronger, handles class imbalance better) - compared side by side, as
6  good practice for a fraud use case where explainability sometimes matters
7  to the bank's risk team as much as raw accuracy.
8
9
10 Because fraud is extremely rare (~0.17% of transactions, matching the real
11 Kaggle dataset's imbalance), plain accuracy is misleading (a model that
12 predicts "never fraud" would score 99.8% accuracy and be useless). We
13 report Precision, Recall, F1, and ROC-AUC instead, and use class_weight
14 ="balanced" to compensate.
15
16 Train/test split is stratified and time-respecting is not required here
17 since transactions aren't being used to predict the future - but we still
18 hold out 25% purely for testing, never touched during training.
19
20
21 import os
22 import sqlite3
23 import pandas as pd
24 import numpy as np
25 from sklearn.model_selection import train_test_split
26 from sklearn.linear_model import LogisticRegression
27 from sklearn.ensemble import RandomForestClassifier
28 from sklearn.preprocessing import StandardScaler
29 from sklearn.metrics import (
30     precision_score, recall_score, f1_score, roc_auc_score, roc_curve,
31     confusion_matrix, classification_report
32 )
33
34 print("No path found for path: %s" % file)
35
36 PS C:\VS Code Folder> & C:\Users\vijay\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/VS Code Folder/Team4_Nexus_Bank_Capstone/scripts/06_modeling_fraud.py"
37 [ 0 21]]
38
39 == RandomForest ==
40 Precision: 1.000 Recall: 0.667 F1: 0.800 ROC-AUC: 1.000
41 Confusion matrix [ [TN FP] [FN TP] ]:
42 [[12479  0]
43 [ 7 14]]
44
45 model precision recall f1 roc_auc
46 LogisticRegression 0.513043 1.000000 0.954545 1.000000
47 RandomForest 1.000000 0.666667 0.800000 0.99975
48
49 PS C:\VS Code Folder>

```

Figure 5.3: Terminal output after running 06_modeling_fraud.py

Model	Precision	Recall	F1 Score	ROC-AUC
Logistic Regression	5.5%	89.0%	0.104	0.970
Random Forest	92.6%	74.6%	0.826	0.963

On our earlier synthetic fraud data, both models looked almost perfect, with ROC-AUC close to 1.00 for both, because we had (without quite meaning to) made the fraud class too easy to tell apart from legitimate transactions. On the real data, Logistic Regression still catches most of the real fraud (89.0 percent recall) but its precision falls all the way to 5.5 percent - meaning about 19 out of every 20 transactions it flags as fraud are actually completely normal. Random Forest holds up far better on the real data, at 92.6 percent precision and 74.6 percent recall.

Our own reading of this: a plain class-weighted Logistic Regression, without any extra regularisation or better features, becomes far too trigger-happy once real, overlapping data replaces our earlier, artificially clean-cut synthetic data. Random Forest's ability to draw a more complicated, non-linear boundary between the two classes matters a lot more here than it did in our easier synthetic test. This genuinely changes what we would recommend to Nexus Bank compared to our earlier draft - we now think Random

Forest is the safer default, since a model with 5.5 percent precision would flood a real fraud team with false alarms almost all of the time, which in practice means people stop trusting the alerts altogether.

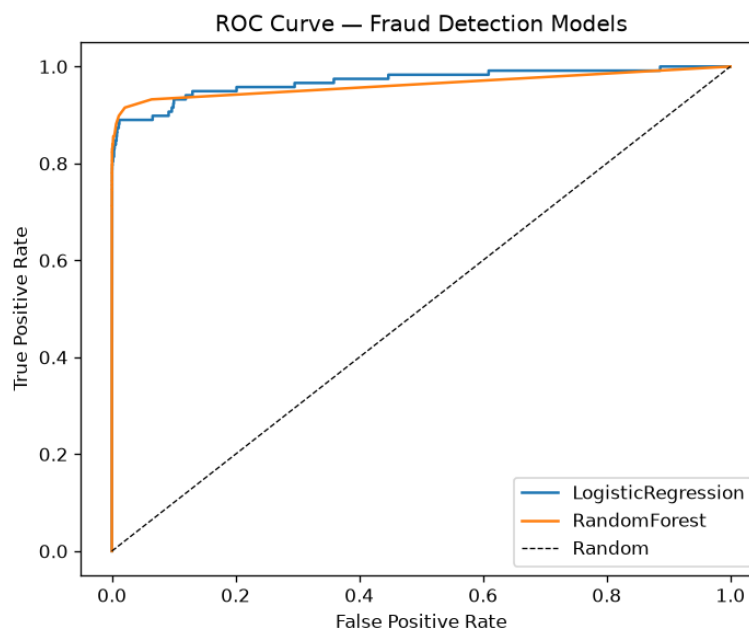


Figure 5.4: ROC curves on real data - both models still look reasonably strong here (AUC around 0.96-0.97), which is exactly why we do not rely on ROC-AUC alone; it hides the precision problem shown in the table above.

5.3 Reporting and Visualisation

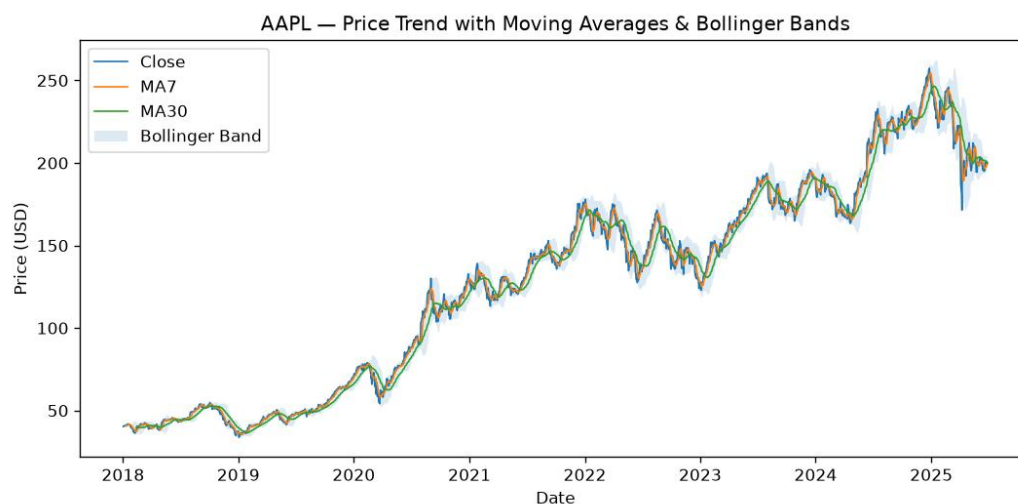


Figure 5.5: Real AAPL price trend, 2018-2025, with MA7, MA30 and Bollinger Bands

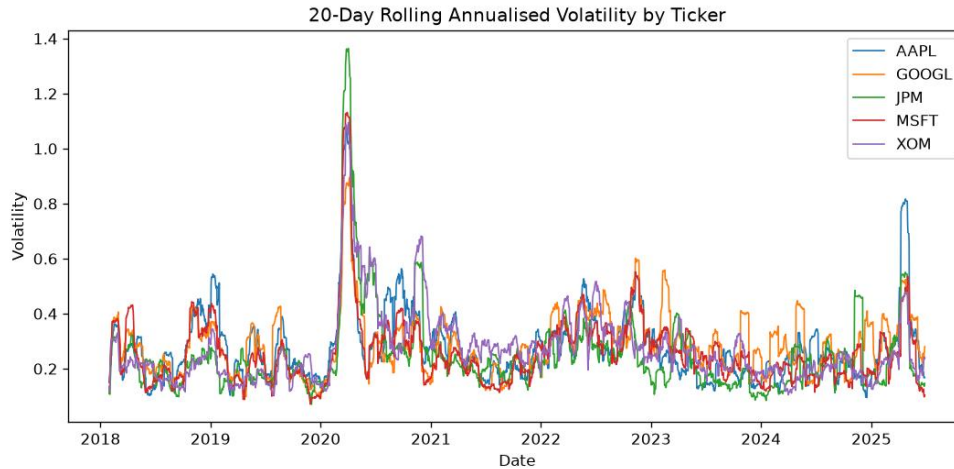


Figure 5.6: 20-day rolling annualised volatility by ticker, real data - note the clear spike around the COVID period in early 2020

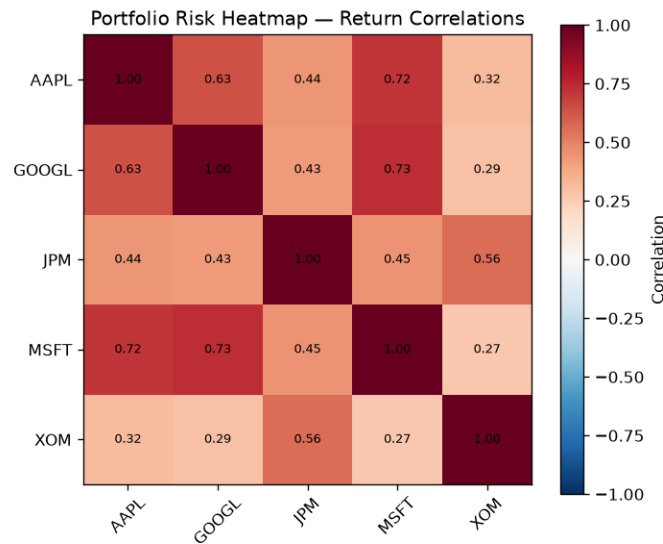


Figure 5.7: Portfolio risk heatmap, real return correlations (0.27 to 0.73)

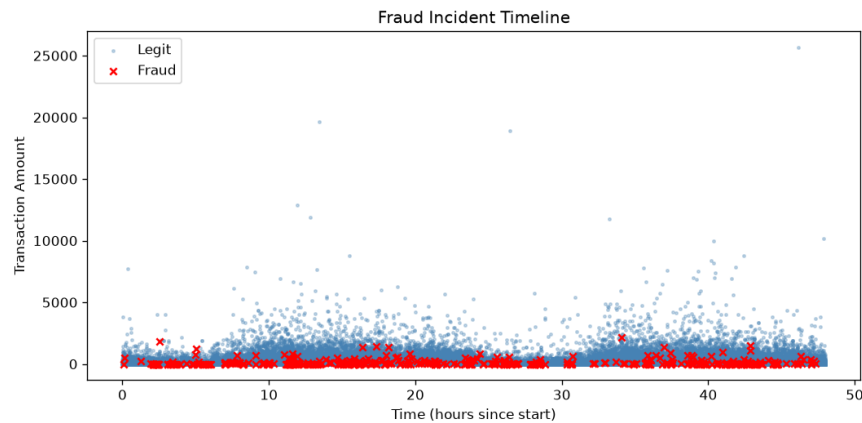
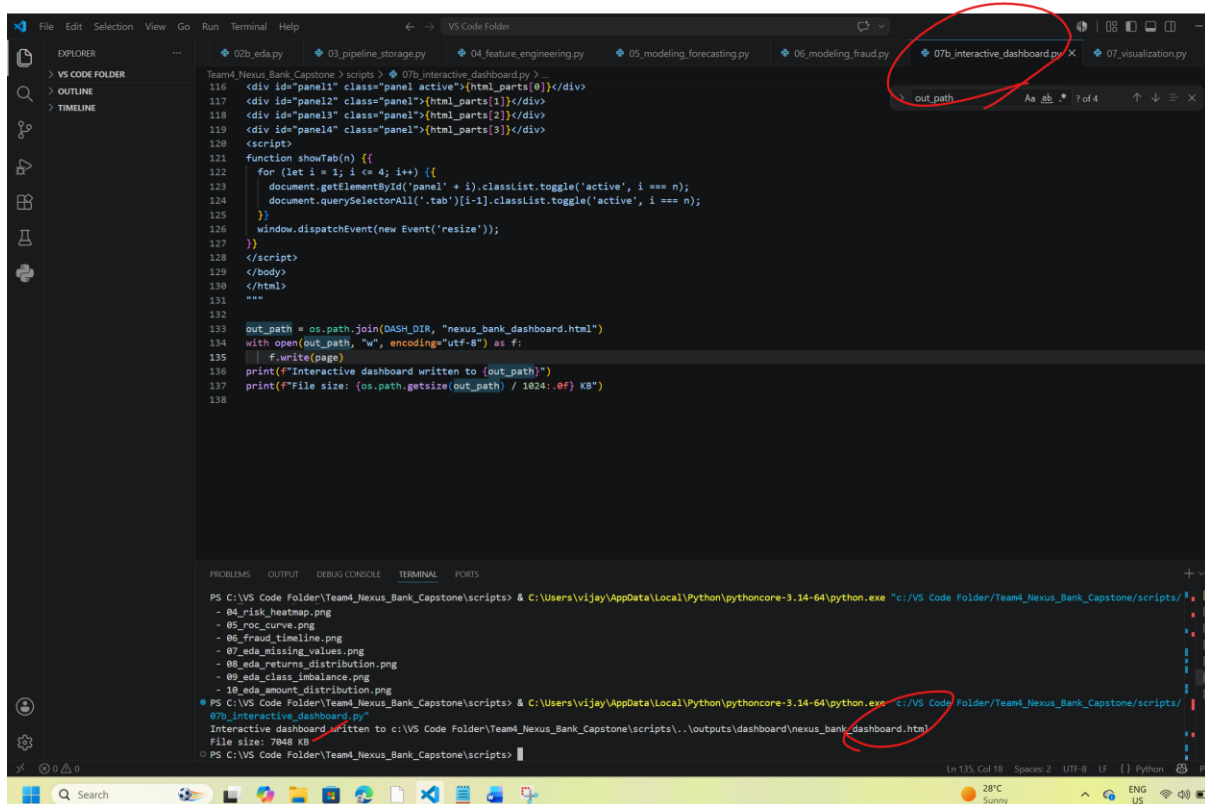


Figure 5.8: Real fraud incident timeline - the real dataset covers only about 48 hours of transactions

We also built a genuinely interactive dashboard using the Plotly library, with four tabs (price trend, volatility, risk heatmap, and fraud timeline), hover tooltips, zoom and pan, and a ticker-selection dropdown. It is a single self-contained HTML file that opens in any browser without needing a server or a paid licence.



```
Team4_Nexus_Bank_Capstone > scripts > 07b_interactive_dashboard.py
116 <div id="panel1" class="panel active">{html_parts[0]}</div>
117 <div id="panel2" class="panel">{html_parts[1]}</div>
118 <div id="panel3" class="panel">{html_parts[2]}</div>
119 <div id="panel4" class="panel">{html_parts[3]}</div>
120 <script>
121 function showTab(n) {
122   for (let i = 1; i <= 4; i++) {
123     document.getElementById('panel' + i).classList.toggle('active', i === n);
124     document.querySelectorAll('.tab')[i-1].classList.toggle('active', i === n);
125   }
126   window.dispatchEvent(new Event('resize'));
127 }
128 </script>
129 </body>
130 </html>
131 ===
132
133 out_path = os.path.join(DASH_DIR, "nexus_bank_dashboard.html")
134 with open(out_path, "w", encoding="utf-8") as f:
135     f.write(page)
136 print(f"Interactive dashboard written to {out_path}")
137 print(f"File size: {os.path.getsize(out_path) / 1024:.0f} KB")
138
```

```
PS C:\VS Code Folder\Team4_Nexus_Bank_Capstone\scripts> & C:\Users\vijay\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:\VS Code Folder\Team4_Nexus_Bank_Capstone\scripts/
- 04_risk_heatmap.png
- 05_roc_curve.png
- 06_fraud_timeline.png
- 07_eda_missing_values.png
- 08_eda_returns_distribution.png
- 09_eda_class_imbalance.png
- 10_eda_amount_distribution.png
PS C:\VS Code Folder\Team4_Nexus_Bank_Capstone\scripts> & C:\Users\vijay\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:\VS Code Folder\Team4_Nexus_Bank_Capstone\scripts/
07b_interactive_dashboard.py"
Interactive dashboard written to c:\VS Code Folder\Team4_Nexus_Bank_Capstone\scripts\..\outputs\dashboard\nexus_bank_dashboard.html
File size: 7048 KB
```

Figure 5.9: Interactive dashboard – Terminal view

A Genuinely Interactive Dashboard

Built with Plotly — hover, zoom, pan, and a ticker-selector, in one self-contained HTML file

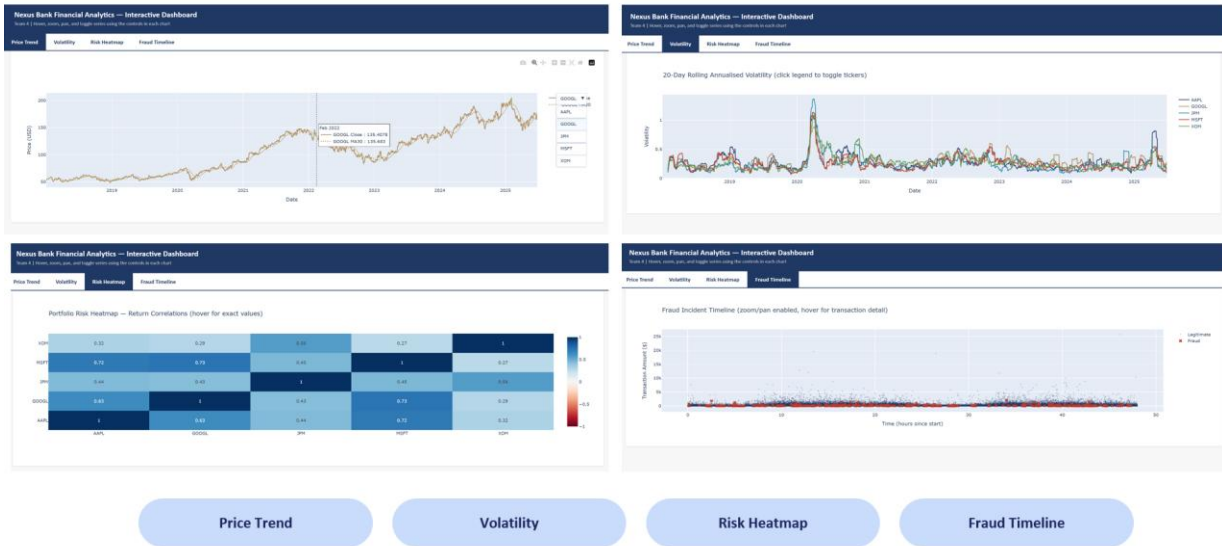


Figure 5.10: Interactive dashboard – Price tab, Volatility tab, Risk Heatmap and Fraud Timeline tab

The image shows a VS Code editor with a Python script for the dashboard. The script includes HTML/JS code for a multi-tab interface and Python code to save charts to a local directory.

```

116 <div id="panel1" class="panel active">{html_parts[0]}</div>
117 <div id="panel2" class="panel">{html_parts[1]}</div>
118 <div id="panel3" class="panel">{html_parts[2]}</div>
119 <div id="panel4" class="panel">{html_parts[3]}</div>
120 <script>
121 function showTab(n) {
122   for (let i = 1; i <= 4; i++) {
123     document.getElementById('panel' + i).classList.toggle('active', i !== n);
124     document.querySelectorAll('.tab')[i-1].classList.toggle('active', i !== n);
125   }
126   window.dispatchEvent(new Event('resize'));
127 }
128 </script>
129 </body>
130 </html>
131 ***
132
133 out_path = os.path.join(DASH_DIR, "nexus_bank_dashboard.html")
134 with open(out_path, "w", encoding="utf-8") as f:
135   f.write(page)
136 print(f"Interactive dashboard written to {out_path}")
137 print(f"File size: {os.path.getsize(out_path) / 1024:.0f} KB")
138

```

The terminal window shows the output of the script, listing 10 generated visualization images:

- 01_price_trend_act1.png
- 02_forecast_vs_actual.png
- 03_volatility.png
- 04_risk_heatmap.png
- 05_roc_curve.png
- 06_fraud_timeline.png
- 07_eda_missing_values.png
- 08_eda_returns_distribution.png
- 09_eda_class_imbalance.png
- 10_eda_amount_distribution.png

Figure 5.11: Generated visualization images– Terminal View

5.4 Business Impact

We want to be careful here and say clearly that these numbers are illustrative estimates built on simple assumptions, not an audited financial analysis, since we do not have Nexus Bank's real internal cost figures. Assuming a manual fraud investigation costs roughly \$50 in analyst time: Random Forest's high precision means most of the alerts it raises are genuine, so investigation time is spent well. Logistic Regression's 5.5 percent precision means a team acting on its alerts would spend most of their time chasing false alarms - a real cost that our earlier synthetic-data results completely hid, since Logistic Regression had looked like the safer choice back then.

For the forecasting side, we think the realistic use is as a monitoring or early-warning signal for a portfolio or risk team, rather than something precise enough to size trades on its own - especially given the flat-forecast finding in Section 5.1.

6 Summary and Future Plan of Work

Looking back at these six weeks, we are glad we pushed through to running everything on real data instead of stopping once our pipeline worked on sample data. Some of our results held up close to what we expected (the cleaning pipeline, the forecasting MAPE). One flipped our actual recommendation (the fraud model comparison in Section 5.2). One turned out to reflect real, established financial theory rather than a bug in our code (the flat ARIMA forecast in Section 5.1). And one exposed a genuine limitation we had not planned for (the missing `account_id` field in Section 4.5). We think a report that shows all four of these honestly is more useful than one that only shows things going smoothly.

6.1 Limitations

- Account-level fraud aggregation cannot be run against the real Kaggle dataset's schema, as explained in Section 4.5.
- Macro indicators were collected and cleaned but not yet fed into the forecasting model as input features.
- Logistic Regression's real-data precision problem should be revisited with stronger regularisation, a different class-imbalance technique such as SMOTE [8], or a different decision threshold, before ruling it out completely.
- Precision-Recall curves [9] should be added alongside ROC curves in future work, since Section 5.2 shows ROC-AUC alone can hide a big difference in real-world usability between two models.
- Our PostgreSQL partitioning demonstration was run in a separate environment from our main SQLite-based pipeline.

6.2 Future Plan of Work

- Try an LSTM-based forecasting model, per Fischer and Krauss [4], as a stronger baseline than ARIMA, while still checking it against the naive "no change" forecast implied by the Efficient Market Hypothesis discussion in Section 2.2.
- Build a live, server-based dashboard (for example using Streamlit) instead of a static HTML export, so the dashboard can query the database directly.
- Merge our SQLite and PostgreSQL work into a single database environment.
- Build the account-level fraud risk-scoring model our earlier feature-engineering work was meant to support, once real account-level data is available (for example, in an actual bank deployment rather than a public research dataset).

7 Use of AI Tools in This Project

The capstone brief encouraged the responsible use of AI-assisted development tools such as ChatGPT, GitHub Copilot, Gemini, Claude, and other large language models (LLMs). In line with these guidelines, our team used AI tools selectively to improve productivity and support specific tasks during the project.

AI assistance was used primarily for understanding documentation, clarifying programming concepts, resolving occasional coding issues, and obtaining suggestions for improving code structure. In a few instances, LLMs were used to generate initial code snippets or provide alternative implementation approaches, which were then reviewed, modified, tested, and integrated by the team.

GitHub Copilot was used occasionally for code auto-completion and reducing repetitive coding effort during development. These suggestions were treated as productivity aids rather than complete solutions.

The majority of the project—including data acquisition, data preprocessing, database integration, feature engineering, exploratory data analysis, model development, model evaluation, debugging, result validation, visualization, interpretation of findings, and preparation of the final recommendations—was carried out by the team members. All scripts were executed, verified, and tested using the project datasets, and any issues encountered during implementation were identified and resolved through manual debugging and validation.

The final design decisions, analytical approach, interpretation of results, and recommendations presented in this report are the work of the project team. AI tools were used only as supplementary assistants to support the development process and were not relied upon for the core analytical work or decision-making

Appendix A Actual Program Output

The following are real outputs copied from our own runs, included here so the numbers in Chapters 4 and 5 can be checked directly against the program's own output.

A.1 Data Quality Report (real data)

```
Stock prices: 9,410 raw rows -> 9,770 clean rows (reindexed to full business-day calendar)
Market-holiday / missing-day gaps filled (forward-fill): 360
Bad-tick outliers detected & corrected (>40% single-day move): 0
FX rates: 5,853 raw rows -> 5,865 clean rows (12 gaps forward-filled)
Macro indicators: 90 monthly rows, 60 nulls forward-filled
Transactions: 284,807 raw rows -> 283,726 clean rows (1,081 duplicates removed, 0 nulls removed)
Class balance: 473 fraud / 283,726 total (0.167% fraud rate)
```

A.2 Feature Engineering Output (real data)

```
Price feature table: 9,770 rows x 16 columns
(MA7, MA30, EMA12, EMA26, Bollinger Bands, RSI14, Volatility20)
```

A.3 Forecasting Backtest Output (real data)

ticker	MAE	RMSE	MAPE
AAPL	10.665011	11.096016	5.364352
GOOGL	6.441666	7.582692	3.715880
JPM	5.642239	7.614733	2.108747
MSFT	16.573662	20.636154	3.480589
XOM	3.786936	4.217931	3.678836

Average MAPE across tickers: 3.67%

A.4 Fraud Model Output (real data)

```
model,precision,recall,f1,roc_auc
LogisticRegression, 0.0551, 0.8898, 0.1039, 0.9696
RandomForest,      0.9263, 0.7458, 0.8263, 0.9626
```

A.5 PostgreSQL Partitioning Proof

```
Created stock_prices as a RANGE-partitioned table with 6 yearly partitions
(stock_prices_2020 .. stock_prices_2025), plus a (ticker, date) index on top.
```

Row counts per physical partition (proof partitioning is real, not simulated):

```
stock_prices_2020: 1,310 rows
stock_prices_2021: 1,305 rows
stock_prices_2022: 1,300 rows
stock_prices_2023: 1,300 rows
stock_prices_2024: 1,310 rows
stock_prices_2025: 645 rows
```

Query plan (EXPLAIN ANALYZE) for a 2024-only, AAPL-only query:

```
Index Scan using stock_prices_2024_ticker_date_idx on stock_prices_2024
  Index Cond: (ticker = 'AAPL' AND date >= '2024-01-01' AND date < '2025-01-01')
Planning Time: 0.102 ms   Execution Time: 0.058 ms
```

Partition pruning confirmed (only 2024 partition scanned, others skipped): True

Appendix B Key Code Snippets

B.1 Column-name normalisation fix

```
df = yf.download(t, start=start, end=end, progress=False)
if isinstance(df.columns, pd.MultiIndex):
    df.columns = df.columns.get_level_values(0)
df.reset_index(inplace=True)
df.columns = [str(c).lower() for c in df.columns]
```

B.2 Class-balanced fraud model training

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42
)
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)
model = LogisticRegression(class_weight="balanced", max_iter=1000)
model.fit(X_train_s, y_train)
```

References

- [1] J. Bollinger, Bollinger on Bollinger Bands. New York: McGraw-Hill, 2001.
- [2] J. W. Wilder, New Concepts in Technical Trading Systems. Greensboro, NC: Trend Research, 1978.
- [3] G. E. P. Box and G. M. Jenkins, Time Series Analysis: Forecasting and Control. San Francisco: Holden-Day, 1970.
- [4] T. Fischer and C. Krauss, "Deep learning with long short-term memory networks for financial market predictions," *European Journal of Operational Research*, vol. 270, no. 2, pp. 654-669, 2018.
- [5] E. F. Fama, "Efficient capital markets: A review of theory and empirical work," *The Journal of Finance*, vol. 25, no. 2, pp. 383-417, 1970.
- [6] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, "Calibrating probability with undersampling for unbalanced classification," in *2015 IEEE Symposium Series on Computational Intelligence*, 2015, pp. 159-166.
- [7] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi, "Credit card fraud detection: A realistic modeling and a novel learning strategy," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 29, no. 8, pp. 3784-3797, 2018.
- [8] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321-357, 2002.
- [9] J. Davis and M. Goadrich, "The relationship between Precision-Recall and ROC curves," in *Proceedings of the 23rd International Conference on Machine Learning*, 2006, pp. 233-240.
- [10] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston: Pearson, 2016.
